

Chapitre V
Les requêtes SQL(Structured query Langage)

Mme. KASSE
Université Alioune Diop de Bambey(UADB)

Web 2421: Base de données
Second semestre 2017

Objectif du cours:

- **Spécifier le schéma relationnel avec SQL(LDD SQL).**

  ❖Niveau conceptuel(schéma , contraintes, gâchettes/trigger)

  ❖Niveau interne(chaque SGBD contient des commandes particulières pour spécifier l'organisation physique).

  ❖Niveau externe(view, privilèges d'accès)

- **Lire les données dans une base de données(Requêtes SQL).**

- **Mettre à jour les données de la base(LMD)**

- **Spécifier les vues et les privilèges d'accès aux tables**

Prérequis:

❖Compréhension du processus de conception de base de données.

❖ Bonne maitrise du modèle conceptuel et du modèle relationnel

❖ Connaissance en algèbre relationnel

Logiciel: Wamp server sous windows

Bibliographie:

1. Livre « Système de gestion de Base de données par l'exemple » 3 eme édition de Robert Godin.
2. http://hoffmannmathias.free.fr/documents/donnees/general_bases_de_donnees/cours%20sql.pdf.

Echéancier

Semaines	Contenu du cours
Semaine 1	Rappel sur la normalisation et correction du devoir+ un exercice sur la normalisation.
Semaine 2	Spécification du schéma relationnel avec SQL et installation d'un SGBD + TP+ travail1 a rendre.
Semaine 3	Remise du travail1+ Requêtes SQL + TP + un travail2 a rendre
Semaine 4	Remise du travail2 +Opération de mises a jour + TP+un travail3 a rendre
Semaine 5	Remise du travail2 +Niveau externe du schéma en SQL+ TP+projet

Pondération:

Chaque travail représente 10% de la note finale,
le projet représente 30% et enfin
un examen final qui représente 40%

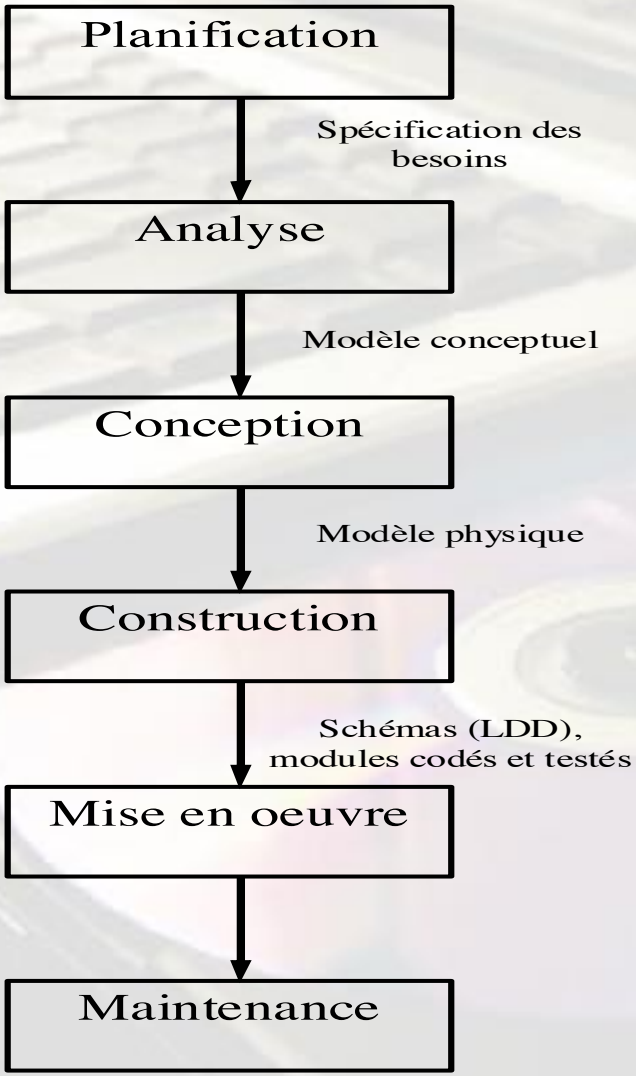
Rappel

Base de données: Ensemble de données possédant les caractéristiques suivantes:

- ➢Fortement structurées
- ➢Persistantes
- ➢Structure définie dans un schéma
- ➢Gérées par un SGBD

SGBD: logiciel spécialisé pour la gestion de Base de données(Oracle, IBM, MySQL, etc). Il permet d'enregistrer, de rechercher, de modifier des donnees dans une base de données.

Processus de conception de base de données



Normalisation

Théorie de normalisation: Comment résoudre certains problèmes de redondance de données.

Décomposition sans perte: Une décomposition binaire d'une table T en deux tables T_1 et T_2 est sans perte si $T = T_1 \bowtie T_2$

Dépendance fonctionnelle(DF): Dans une relation R, on dit qu'il y a DF entre un ensemble d'attributs A et un ensemble d'attributs B dans la table T si , pour toutes instances de la table, lorsque deux lignes ont les mêmes valeurs pour l'ensemble des colonnes de A, elles ont nécessairement les mêmes valeurs pour l'ensemble des colonnes de B.

Exemple Pour la table Vente:

noClient → nomClient, noTelephone
noCommande→noClient,dateCommande
noCommande,noArticle→quantite
noArticle→description, priUnitaire

Normalisation

Déterminant d'une dépendance fonctionnelle(DF):
La partie gauche d'une DF est appelée déterminant.

Dépendance fonctionnelle triviale: partie droite incluse dans la partie gauche.

Exemple: noCommande, noArticle, description→ noCommande, noArticle

Dépendance fonctionnelle complètement non triviale: aucune colonne de la partie droite n'apparait dans la partie gauche.

Dépendance fonctionnelle pleine: partie gauche minimale.

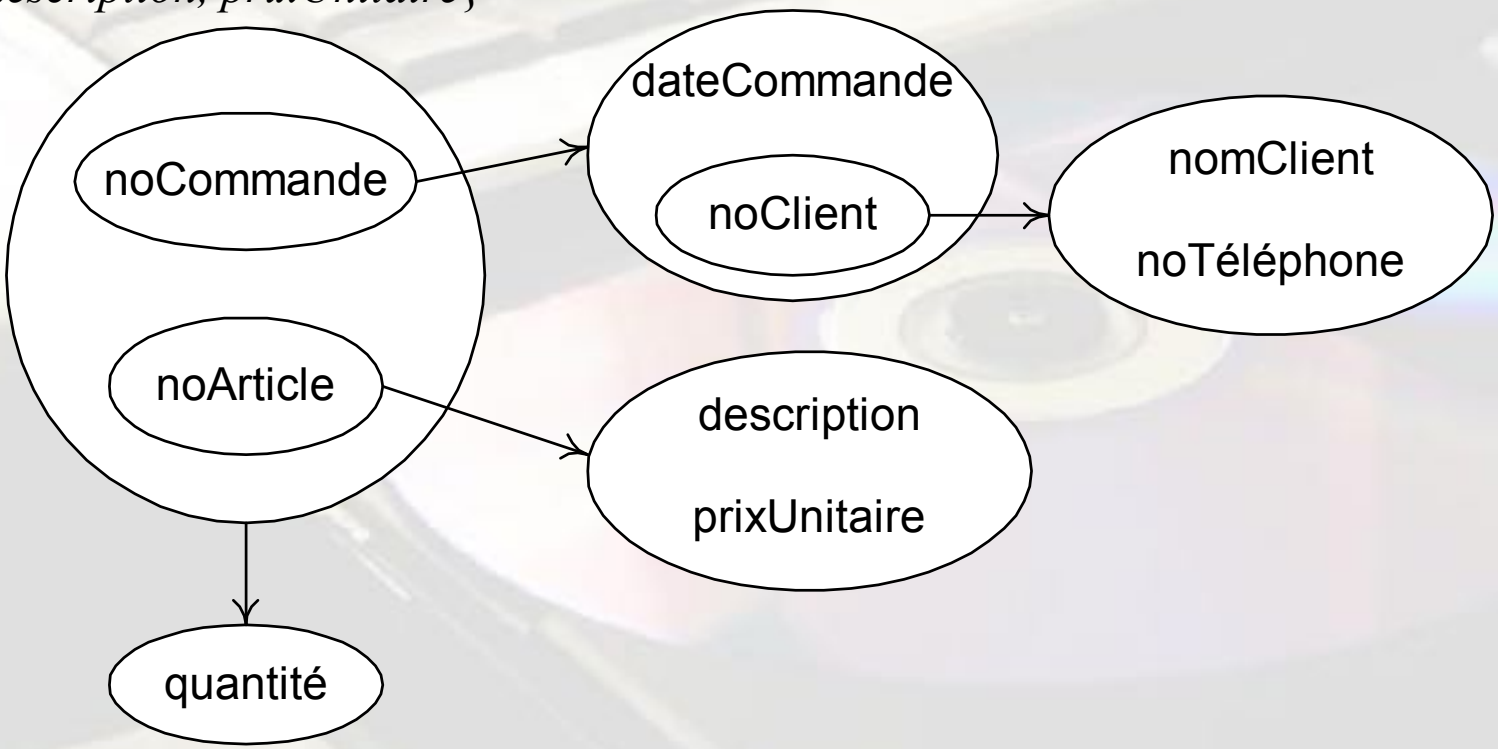
Axiome d'amstrong:

- ➢Réflexivité
- ➢Augmentation
- ➢transitivité

Normalisation

Représentation des dépendances fonctionnelles par le diagramme a bulles.

D = {*noClient* → *nomClient*, *noTéléphone* ; *noCommande* → *noClient*, *dateCommande* ; *noCommande, noArticle* → *quantité* ; *noArticle* → *description, prixUnitaire*}



Normalisation

Fermeture d'un ensemble d'attribut A^+ : $\{noClient\}^+ = \{noClient, nomClient, noTéléphone\}$.

Dérivabilité: D|=A→B est dérivable ssi B inclus dans A^+

Couverture minimale de D:

✓Min(D) est équivalent a D

✓Les dépendances de min(D) sont élémentaires

✓Aucune dépendance de min(D) n'est déductible des autres.

Normalisation

D = {*noClient* → *noClient*, *nomClient*, *noTéléphone* ;
noCommande → *noClient*, *noTéléphone* ;
noCommande → *dateCommande* ;
noCommande, noArticle, noClient → *quantité*, *noArticle*;
noArticle → *description, prixUnitaire* ;
noCommande → *nomClient* }

1. Éclater les DF dont la partie droite est composée de plus d'un attribut.
2. Éliminer les dépendances triviales.
3. Rendre les dépendances pleines.
4. Éliminer les dépendances déductibles des autres.

Clé Candidate: A est clé candidate ssi C(l'ensemble des colonnes)=A^+ et il n'ya pas de B inclus dans A tel que C=B^+

Normalisation

Forme normale de Boyce-codd(FNBC): toute DF non triviale et pleine

A→B, A est une clé candidate de T.

Troisième forme normale: toute DF non triviale et pleine

A→B, soit A est une clé candidate de T ou B fait partie d'une clé candidate.

Le langage SQL

LANGAGE DE DEFINITIONS DE DONNEES(LDD ou DDL)

- CREATE TABLE ..
- ALTER TABLE
- DROP TABLE

LANGAGE DE MANIPULATION DE DONNEES(LMD ou DML)

- SELECT
- INSERT
- UPDATE
- DELETE

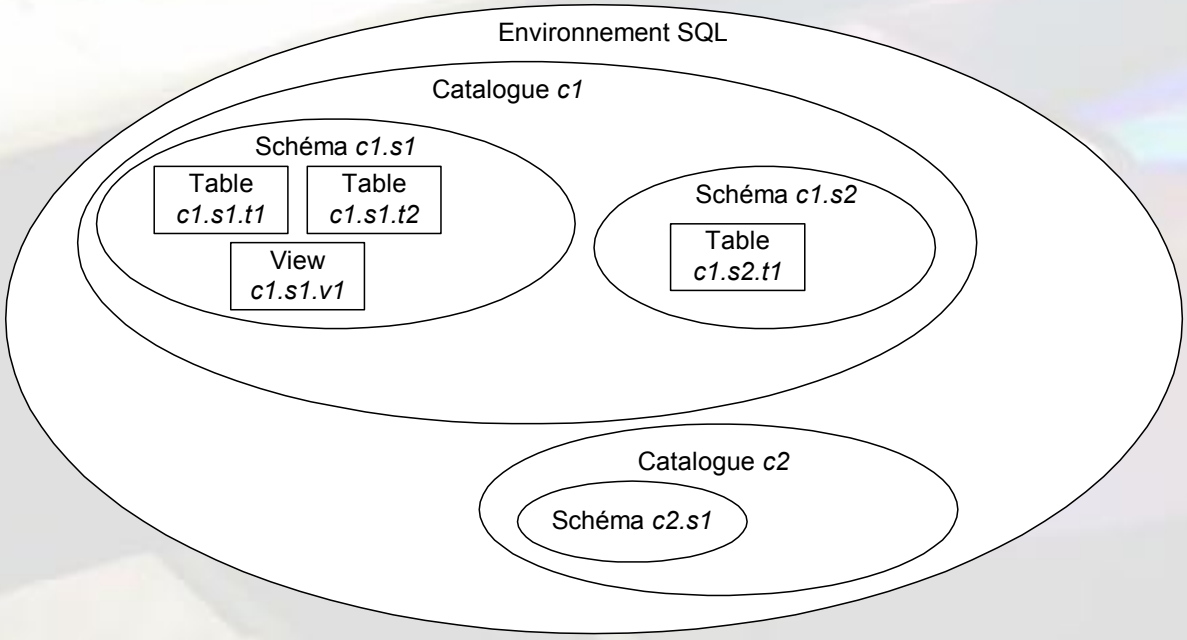
LANGAGE DE CONTROLE DE DONNEES(LCD ou DCL)

- GRANT et REVOKE
- BEGIN et END TRANSACTION
- COMMIT et ROLLBACK

Le langage SQL

Schéma SQL: Représente un espace de nom prive pour une collection d' élément de définition de données(Table, View Domaine, etc).

Catalogue SQL: Un catalogue est un espace de nom prive regroupant un ensemble de schéma.



TYPES SQL

Type	Description	Taille
INTEGER	Type des entiers relatifs	4 octets
SMALLINT	Idem.	2 octets
BIGINT	Idem.	8 octets
FLOAT	Flottants simple précision	4 octets
DOUBLE PRECISION	Flottants double précision	8 octets
REAL	Synonyme de FLOAT	4 octets
NUMERIC (*M*, *D*)	Numérique avec précision fixe.	*M* octets
DECIMAL (*M*, *D*)	Idem.	*M* octets
CHAR(*M*)	Chaînes de longueur fixe	*M* octets
VARCHAR(*M*)	Chaînes de longueur variable	*L*+1 avec $L \leqslant M$
BIT VARYING	Chaînes d'octets	Longueur de la chaîne.
DATE	Date (jour, mois, an)	env. 4 octets
TIME	Horaire (heure, minutes, secondes)	env. 4 octets
DATETIME	Date et heure	8 octets
YEAR	Année	2 octets

CLE D'UNE TABLE

Clé d'une table:

Une clé est un attribut (ou un ensemble d'attributs) qui identifie(nt) de manière unique un tuple d'une relation. Il peut y avoir plusieurs clés mais l'une d'entre elles doit être choisie comme clé primaire.

Clé primaire:

la clé primaire est la clé utilisée pour référencer une ligne et une seule d'une table, elle est spécifiée avec l'option **PRIMARY KEY**.

Clé étrangère:

La norme SQL ANSI permet d'indiquer quelles sont les clés étrangères dans une table ie les attributs qui font référence à une ligne dans une autre table. On peut spécifier les clés étrangères avec l'option **FOREIGN KEY**.

CREATION D'UNE TABLE

Les objets du schéma sont:

créés avec ***create*,**

modifiés avec ***alter*** et

détruits avec ***drop***,

alors que les données, instances du schéma sont créées, modifiées et détruites avec, respectivement, insert, update et delete

Création du schéma d'une table en SQL(Create table): La commande principale est: ***create table***.

CREATE TABLE nomDeLaTable

(TypeDeColonne,

[,specificationDeColonne]….

[,specificationDeContrainte]…)

CREATION D'UNE TABLE

Spécification de colonnes:

nomColonne [type|domaine] [DEFAULT valeurDeDéfaut]
[NULL | NOT NULL] [UNIQUE | PRIMARY KEY]
[REFERENCES nomTable[listeColonnes]]
[[CONSTRAINT nomContrainte] CHECK (conditionSQL)]

Spécification de contraintes

[CONSTRAINT nomContrainte]
{PRIMARY KEY listeColonnes|
FOREIGN KEY listeColonnes REFERENCES nomTable[listColonnes]
[MATCH{{PARTIAL|FULL}]
[ON DELETE{No ACTION|CASCADE|SET NULL|SET DEFAULT]}]
[ON UPDATE{No ACTION|CASCADE|SET NULL|SET DEFAULT]}]
|CHECK(conditionSQL)}

CREATION D'UNE TABLE

Exemple:

CREATE TABLE Client (id INTEGER NOT NULL ,
nom VARCHAR(15) NOT NULL,
noTéléphone VARCHAR(15) ,
PRIMARY KEY (id)).

Le NOT NULL: oblige a toujours indiquer une valeur.
DEFAULT : attribue une valeur par défaut
PRIMARY KEY : indique si cette colonne est considérée comme clé primaire pour un index.

CREATION D'UNE TABLE(clé étrangère)

Exemple:

```
CREATE TABLE Commande
(noCommande      INTEGER                    NOT NULL,
 dateCommande    DATE                       NOT NULL,
 noClient        INTEGER                    NOT NULL,
 PRIMARY KEY     (noCommande),
 FOREIGN KEY     (noClient) REFERENCES Client)
```

CREATION D'UNE TABLE

Énumération: CHECK

La norme SQL ANSI comprend une option CHECK(condition) pour exprimer des. La condition elle-même peut être toute expression suivant la clause WHERE dans une requête SQL.

Les contraintes les plus courantes sont celles consistant à restreindre un attribut à un ensemble de valeurs.

Exemple:

create table Article(noArticle integer not null, description varchar(20), check (description IN("lait","sucre","riz")), quantite integer, primary key (noArticle));

INTEGRITE REFERENTIELLE

Dans le cas d'une entité faible, on décide en général de détruire le composant quand on détruit le composé.

Exemple: quand on détruit un client, on veut également détruire les commandes qu'il a effectue; quand on modifie la clé d'un client, on veut répercuter la modification sur toutes se commandes.

CREATE TABLE Commande

(noCommande INTEGER NOT NULL,
dateCommande DATE NOT NULL,
noClient INTEGER NOT NULL,
PRIMARY KEY (noCommande),
FOREIGN KEY (noClient) REFERENCES Client) ON DELETE CASCADE
ON UPDATE CASCADE).

SUPPRESSION D'UNE TABLE

DROP TABLE NomTable [RESTRICT| CASCAD]

RESTRICT: Rejet si élément dépendant existe.

CASCADE: supprime élément dépendant.

MODIFICATION DU SCHEMA DE LA TABLE

Modification du schéma:

On a toujours besoin de créer de nouvelles tables, d'ajouter des colonnes ou même modifier leur définition.

La forme générale de la commande permettant de modifier une table est :

ALTER TABLE nomTable

{ADD COLUMN specificationColonne|

DROP COLUMN nomColonne[RESTRICT|CASCADE]|

ADD specificationContrainte

DROP nomContrainte[RESTRICT|CASCADE]|

ALTER nomColonne SET DEFAULT valeurDefaut|

ALTER nomColonne DROP DEFAULT}…

Exemple: Ajouter une colonne âge de type entier strictement positive dans la table client.

INSERTION DES DONNEES DANS UNE TABLE

L'insertion s'effectue avec la commande INSERT dont la syntaxe est la suivante :

INSERT INTO T VALUES (v_1,..., v_n); ou

INSERT INTO T (A_1, A_2, ... A_n) **VALUES** (v_1, v_2, ... v_n)

T est le nom d'une relation, et les A_1, ... A_n sont les noms des attributs dans lesquels on souhaite placer une valeur. *Les autres attributs seront donc à* NULL *(ou à la valeur par défaut)*.

Les v_1, ... v_n sont les valeurs des attributs.

INSERTION DU RESULTAT D'UNE REQUETE

Dans ce cas la partie VALUES ... est remplacée par la requête elle-même.

Exemple : on a créé une table ClientOrange (id, noTelephone) et on souhaite y copier les couples (id, noTelephone) déjà existant dans la table Client.

INSERT INTO Client (id, noTlephone)

SELECT (id, noTlephone FROM Client).

REQUETES SIMPLES SQL

SQL est un langage déclaratif qui permet d'interroger une base de données sans se soucier de la représentation interne (physique) des données, de leur localisation, des chemins d'accès ou des algorithmes nécessaires.

Exemple1: de requête simple: on veut extraire le nom des client de la base.

Select nom

from client

Exemple2: On veut extraire le nom des client qui se nomment Moussa.

Select nom

from client

Where nom=« Moussa »

REQUETES SIMPLES SQL

Les fonctions applicables aux valeurs des attributs sont par exemple les opérations arithmétiques (+,*, /...).

Exemple: on veut afficher la moitié de la quantité livrée de la livraison numéro 100 .

```
select quantiteLivree/2
FROM DetailLivraison
Where noLivraison=100;
```

Renommage

Les noms des attributs sont par défaut ceux indiqués dans la clause SELECT, même quand il y a des expressions complexes. Pour renommer les attributs, on utilise le mot-clé **AS**.

```
select quantiteLivree/2 as quantiteReduite
FROM DetailLivraison
Where noLivraison=100;
```

REQUETES SIMPLES SQL

Pour éviter d'obtenir deux tuples identiques, on peut utiliser le mot-clé **DISTINCT.**

Exemple :

```
SELECT DISTINCT nom
FROM Client;
```

Tri du résultat

Il est possible de trier le résultat d'une requête avec la clause **ORDER BY** suivie de la liste des attributs servant de critère au tri.

Exemple :

```
SELECT *
FROM Client
ORDER BY nom;
```

Pour trier en ordre descendant, on ajoute le mot-clé **DESC** après la liste des attributs.

REQUETES SIMPLES SQL

Pour avoir toutes les lignes on omet la clause **WHERE**, et pour avoir toutes les colonnes, on peut au choix lister tous les attributs ou utiliser le caractère '*' qui a la même signification.

La clause WHERE

Dans la clause **WHERE**, on spécifie une condition booléenne portant sur les attributs des relations du **FROM**. On utilise pour cela de manière standard le **AND**, le **OR**, le **NOT** et les parenthèses pour changer l'ordre de priorité des opérateurs booléens.

REQUETES SIMPLES SQL

Les opérateurs de comparaison sont ceux du Pascal: <,<=,>,>=,= et <> pour la différence. Pour exprimer la différence(!= est également possible).

Exemple: les *Articles* dont le prix est compris entre 12 et 25

```
SELECT noArticle, description
FROM Article
WHERE  prixUnitaire > 12 AND prixUnitaire < 25
```

Pour obtenir une recherche par intervalle, on peut également utiliser le mot-clé **BETWEEN**.

```
SELECT noArticle, description
FROM Article
WHERE  prixUnitaire between 12 and 25
```

REQUETES SIMPLES SQL

Chaînes de caractères

SQL fournit des options pour les recherches par motif à l'aide de la clause **LIKE**.

Le caractère '_' ou ' **?**' désigne n'importe quel caractère, et le '**%**' n'importe quelle chaîne de caractères.

Exemple: chercher tous les clients dont le nom se termine par un 'a'.

```
SELECT nom
FROM client
WHERE nom LIKE '%a';
```

REQUETES SUR PLUSIEURS TABLES

Tout au long de cette partie nous allons travailler sur la base Vente(extraite du livre « ROBERT GODIN ») dont le modèle relationnel est le suivant:

Client(noClient, nomClient, noTelephone)
Commande(noCommande, dateCommande, noClient)
Article(noArticle,description, prixUnitaire, quantiteEnStock)
LigneCommange(noCommande, noArticle, quantite)
Livraison(noLivraison, dateLivraison)
DetailLivraison(noLivraison, noCommande, noArticle, quantiteLivree).

REQUETES SUR PLUSIEURS TABLES

Pour exprimer la jointure, on donne simplement la liste des tables concernées dans la clause FROM, et on exprime les critères de rapprochement entre ces tables dans la clause WHERE.

Exemple: On veut les informations sur les clients et les commandes qu'ils ont effectuées.

Select Client.noClient, nomClient, telephone,noCommande,dateCommande
From Client,Commande
Where Client.noClient=Commande.noClient

REQUETES SUR PLUSIEURS TABLES

Renommage(as):

On peut réécrire la requête précédente comme suit:

Select c.noClient, nomCient, telephone, noCommande,dateCommande

From Client cl, Commande co

Where cl.noClient=co.noClient.

Eemple : Clients qui ont commande une plante d'herbe a puce.

On peut exprimer la jointure avec le mot clé JOIN.

Exemple: Pour la même requête on a:

Select *

From Client NATURAL JOIN Commande(SQL2)

REQUETES SUR PLUSIEURS TABLES

Auto-Jointure:

Techniquement, on résout le problème en SQL en utilisant deux synonymes distincts.

Exemple: Les clients qui ont le même numéro de téléphone.

Select cl1.noClient, cl2.noClient

From Client cl1, Client cl2

Where cl1.notelephone==cl2.notelephone and cl1.nomClient!=cl2.nomClient.

REQUETES SUR PLUSIEURS TABLES

Jointure externe:

Les clients et leurs commandes incluant les informations sur les Clients qui n'ont pas placé de Commande.

```
Select *
From client Natural Left Outer Join Commande
```

Operations : Union Intersection et la différence

Table *Employé*		
codeEmployé	**nomEmployé**	**noTéléphone**
CASD1	Dollard Tremblay	(888)888-8888
PIOY1	Yan Piochuneshot	911
LAFH1	Yvan Lafleur	(111)111-1111
HASC1	Jean Leconte	(666)666-6666

On construit deux requêtes dont les résultats ont le même nombre de colonnes et mêmes types d'attributs, et on les relie par un des mot-clé UNION, INTERSECT ou EXCEPT.

Exemple: Noms et numéros de téléphone des Employés qui sont aussi des Clients.

```
Select nomClient as nomPersonne, notelephone
From Client
INTERSECT
Select nomEmployer as nomPersonne, notelephone
From Employer
```

Expressions générale sur les colonnes

Eemple1: La liste des noArticle avec le prixUnitaire avant et après inclusion de la taxe de 15%.

Select noArticle, prixUnitaire, prixUnitaire*1.15 as prixPlusTaxe
From article.

Eemple2:Les Articles dont le prixUnitaire incluant la taxe de 15% est inférieur à $16.00

Expressions générale sur les colonnes

Exemple1: La liste des noArticle avec le prixUnitaire avant et après inclusion de la taxe de 15%.

Select noArticle, prixUnitaire, prixUnitaire*1.15 as prixPlusTaxe
From article.

Exemple2:Les Articles dont le prixUnitaire incluant la taxe de 15% est inférieur à $16.00

Fonction d'agrégation

Il est possible d'utiliser des fonctions *f* d'agrégation dans le résultat d'une sélection.

Syntaxe :

SELECT *f* (expression)

FROM ...

où *f* peut être COUNT: nombre de tuples

SUM: somme des valeurs d'une colonne

AVG: moyenne des valeurs d'une colonne

MAX: maximum des valeurs d'une colonne

MIN: minimum des valeurs d'une colonne

Pour COUNT, on peut aussi utiliser COUNT(*)

Fonction d'agrégation

- Résultats de Modou

Matière	*Coef*	*Note*
Maths	4	15
Sc Nat	3	9
Sc Phy	3	12
Français	2	13
Sc Hum	2	11
Anglais	1	10
Sport	1	12

Exemple:

1**. Quelle est la meilleur note?**

SELECT **MAX**(*Note*) FROM *Résultats*

2. Quelle est la plus mauvaise note ?

SELECT **MIN**(*Note*) FROM *Résultats*

Quelle la somme pondérée des notes ?

SELECT **SUM**(*Note*Coef*) FROM *Résultats*

Quelle est la moyenne (pondérée) de Modou ?

SELECT **SUM**(*Note*Coef*)/***Sum***(*Coef*) FROM *Résultats*

Dans combien de matières Modou a-t-il eu plus de 12 ?

SELECT **COUNT(*)** FROM *Résultats* WHERE *Note* > 12

La clause GROUP BY

- **Remarque importante** : On ne peut pas utiliser simultanément dans la clause SELECT des fonctions d'agrégation et des noms d'attributs (sauf dans le cas d'un **GROUP BY**)
- A condition de respecter cette règle, on peut utiliser les ordres SQL les plus complexes.
- **Exemple**: Le nombre de Commandes passées par chacun des Clients qui ont passé au moins une Commande
  - SELECT noClient,COUNT(*)as nombreCommande
  - FROM Commande
  - Group by noClient

La clause HAVING

On peut faire porter des conditions sur les groupes avec la clause HAVING. La clause WHERE ne peut exprimer des conditions que sur les tuples pris un à un.

Exemple1 : Quels sont les coefficients auxquels participe une seule matière ?

```
SELECT coef
FROM Résultats
GROUP BY coef
HAVING count(*)=1;
```

Exemple2 : le nombre de Commandes passées par chacun des Clients qui ont passé deux Commandes ou plus après le 02/06/2000.

Les sous-requêtes

L'utilisation des sous-requêtes est d'offrir une alternative syntaxique à l'expression de jointures

Exemple1 : Les Clients qui ont passé au moins une Commande

```
SELECT *
FROM Client
WHERE noClient IN(SELECT noClient
                  From Commande)
```

Exemple2 : les LigneCommandes pour lesquelles au moins une Livraison a été effectuée.

Les sous-requêtes

Le mot-clé **IN** exprime clairement la condition d'appartenance de noClient à la relation formée par la requête imbriquée. On peut remplacer le IN par un simple '=' *si on est sûr que la sous-requête ramène un et un seul tuple*.

Exemple : les Commandes du Client Jean Leconte

```
Select *
From Commande
Where noClient =(Select noClient
                 From Client
                  Where nomClient=« Jean Leconte)
```

Les sous-requêtes

Voici les conditions que l'on peut exprimer sur une relation construite avec une requête imbriquée.

1. EXISTS T Renvoie True si T n'est pas vide, Fasle sinon.
2. t **IN** T où t est un tuple dont le type est celui de T. True si t appartient à T, False sinon.
3. V compare **ANY** T, où cmp est un comparateur SQL(<,>,=, etc). Renvoie True si la comparaison avec au moins un des tuples de la relation unaire T renvoie True.
4. V cmp **ALL** T. Renvoie True si la comparaison avec tous les tuples de la relation unaire T renvoie True.

Toutes ces expressions peuvent être préfixer par **NOT** pour obtenir la négation.

Les sous-requêtes

Exemple1: Les Clients qui ont passé au moins une Commande.

```
Select *
From Client
Where EXISTS(Select *
             From Commande
             Where noClient=Client.noClient).
```

On peut tester un double avec le mot clé **NOT UNIQUE**.

Exemple2: Clients qui ont passé au moins deux Commandes.

```
Select *
From Client
Where EXISTS(Select *
             From Commande
             Where noClient=Client.noClient).
```

Les sous-requêtes

Exemple1: Commandes passées après la dernière Livraison.

```
Select *
From Commande
Where dateCommande > ALL(Select dateLivraison
                         From Livraison)
```

Exemple2: Commandes passées après au moins une des livraisons.

Division

$T_1 \subseteq T_2$ ssi NOT EXISTS (T_1 EXCEPT T_2)

Exemple: Les commandes qui incluent tous les articles dont le prixUnitaire est 10.99.

```
Select noCommande
From Commande where NOT EXISTS
        ((Select noArticle
          From Article
          Where prixUnitaire=10.99) EXCEPT
              (Select noArticle
               From LigneCommande
               Where noCommande=commande.noCommande)
        )
```

Destruction

La destruction s'effectue avec la clause **DELETE** dont la syntaxe est :

DELETE FROM T

WHERE condition

T est bien entendu la table, et condition est toute condition valide pour une clause WHERE.

Exemple: Supprimer toutes les lignes de la table *Client*

DELETE FROM Client

Destruction

Exemple1 : destruction de tous les clients dont le nom commence par ’M’.

```
DELETE FROM Client
WHERE nom LIKE ’M%’
```

Eemple3: Supprimer les clients qui n’ont pas passe de commande.

Modification

La modification s'effectue avec la clause **UPDATE**.

UPDATE T **SET** A1=v1, A2=v2, ... An=vn

WHERE condition

T est la relation, les Ai sont les attributs, les vi les nouvelles valeurs et condition est toute condition valide pour la clause WHERE.

Exemple1: augmenter les notes en math de Modou d'un point.

```
UPDATE Résultats
SET note = note+1
WHERE Matière = 'Maths'
```

Exemple2: Augmenter tous les *prixUnitaires* des *Articles* de 10%

Exemple3: changer le prix unitaire et la quantité (12.99 et 5) pour l'article numéro 10.

Les Vues

Une requête produit toujours une relation. Cela suggère la possibilité d'ajouter au schéma des tables 'virtuelles' qui ne sont rien d'autres que le résultat de requêtes stockées. De telles tables sont nommées des *vues*. On peut interroger des vues comme des tables stockées.

Une vue n'induit aucun stockage puisqu'elle n'existe pas physiquement, et permet d'obtenir une représentation différente des tables sur lesquelles elle est basée.

Création d'une vue:

La syntaxe de création d'une vue est très simple:

CREATE VIEW <nom-vue> **AS** <requête>

Les Vues

Exemple: on peut créer une vue qui ne Contient que les articles dont le prix unitaire est inferieur à 15.

```
CREATE VIEW articleMoinsChers AS
Select noArticle, description, prixUnitaire
 FROM article
 WHERE prix < 15
```

Mise à jour d'une vue

Les règles définissant les vues modifiables sont très strictes.

1. La vue doit être basée sur une seule table.
2. Toute colonne non référencée dans la vue doit pouvoir être mise à NULL ou disposer d'une valeur par défaut.
3. On ne peut pas mettre-à-jour un attribut qui résulte d'un calcul ou d'une opération.

Exemple1: Supprimer l'article numéro 20 dans la vue articleMoinsChers

Insérez dans la vue articleMoinsChers l'article(200, « viagra» ,50.99)

On détruit une vue avec la syntaxe **DROP VIEW nomVue**

Renommer les colonnes d'une vue

Create view TotalCommande (noCommande, total, totalPlusTaxe) as select noCommande , sum(prixUnitaire+quantite), sum(prixUnitaire*quantite*1.15)

From LigneCommande L, Article A

Where L.noArticle=A.noArticle

GroupBy noCommande

WHITH CHECK OPTION

Permet d' éviter les mises a jours incohérentes par rapport a la sémantique de la vue.

CREATE VIEW articleMoinsChers **AS**

Select noArticle, description, prixUnitaire

FROM article

WHERE prix < 15

WHITH CHECK OPTION

Insérez dans la vue articleMoinsChers l'article (200, « viagra» ,50.99) :

WHITH CASCADED OPTION

Permet de vérifier que la sémantique de la vue est respectée par les mises a jour ainsi que la sémantique de toutes les vues utilisées directement ou indirectement par la vue.

Exemple: CREATE VIEW articleMoyens **AS**

Select noArticle, description, prixUnitaire

FROM articleMoinsChers

WHERE prix > 12

WHITH CASCADED CHECK OPTION

Insérez l'article(300, «Bissap», 50) dans la vue articleMoinsChers.

WHITH LOCAL CHECK OPTION

Limite la vérification a la vue elle-même sans tenir compte des vues sous-jacentes.

Exemple: CREATE VIEW articleMoyens **AS**

Select noArticle, description, prixUnitaire

FROM articleMoinsChers

WHERE prix > 12

WHITH LOCAL CHECK OPTION

Insérez l'article(300, «Bissap», 50) dans la vue articleMoinsChers.

Ajout d'un utilisateur

La commande **CREATE USER** crée un nouveau compte MySQL. Pour l'utiliser, vous devez avoir les droits de CREATE USER ou les droits d'INSERT dans la table de droits de la base mysql. Le compte peut recevoir un mot de passe avec la clause **IDENTIFIED BY.**

Syntaxe:CREATE USER '***user'@'localhost'*** IDENTIFIED BY ***password*** .

Exemple:

create user 'fatou'@'localhost' identified by 'ndiaye';

Ajout d'un utilisateur

Pour se connecter avec l'utilisateur créé,

on exécute la commande

mysql –u user –p password sous l'invite de commande DOS.

Exemple

Sous DOS on exécute la commande

Mysql -u 'fatou' –p

password>ndiaye

Grant et Revoke

Les commandes **GRANT** et **REVOKE** permettent à l'administrateur système de donner ou retirer des droits aux utilisateurs.

Exemple on donne à fatou les droit de sélectionner la table client de la base vente:

Grant select on vente.client to fatou;

Pour lui retirer ce droit:

Revoke select on vente.client from fatou;

On peut lui donner ou retirer tous les droits:

Grant all privileges on vente.* to fatou

Revoke all privilges on vente.* from fatou

REFERENCES

- http://lig-membres.imag.fr/plumejeaud/NFP107/cours_PhilippeRigaux_bd.pdf
- Livre: Système de base de données par l'exemple(Robert Godin)